



FleetOps

PRODUCTION DEPLOYMENT GUIDE

Taking FleetOps live on the internet

Docker images on Docker Hub, running on an Oracle Cloud (OCI) VM, behind HTTPS with automatic certificates.

VERSION	1.0
DATE	3 October 2026
APPLIES TO	FleetOps (naventra-platform) main branch
AUDIENCE	Developers / DevOps deploying FleetOps

Contents

1. How the deployment works	4
2. Images and containers	4
2.1 Images you build and publish (2)	4
2.2 Official images pulled as-is (3)	5
2.3 Build-only images (never run in production)	5
2.4 Local vs production	5
3. Architecture	6
3.1 Request flow	6
3.2 Ports	6
4. Prerequisites and costs	7
4.1 Recommended VM size	7
5. Phase 1 - Prepare a release	7
6. Phase 2 - Publish the images to Docker Hub	8
6.1 One-time Docker Hub setup	8
6.2 Build for both CPU types and push	8
7. Phase 3 - Create the Oracle Cloud infrastructure	9
7.1 Network (VCN)	9
7.2 Compute instance	9
7.3 Reserve the public IP	9
7.4 Connect	9
8. Phase 4 - Prepare the server	10
8.1 Updates and basics	10
8.2 Open the host firewall (important on OCI)	10
8.3 Install Docker Engine and the Compose plugin	10
9. Phase 5 - Point your domain at the server	11
10. Phase 6 - Deploy FleetOps	11
10.1 Copy the deployment files to the server	11
10.2 Create the secrets file	11
10.3 Log in to Docker Hub on the server	12
10.4 Start the stack	12
10.5 Verify	12
11. Phase 7 - Operating the system	12
11.1 Releasing a new version	12

11.2 Rolling back

12

11.3 Backups

13

11.4 Restoring a backup

13

11.5 Logs, monitoring and housekeeping

13

11.6 Growing later

13

12. Automating releases with GitHub Actions (optional)

14

13. Security checklist

15

14. Troubleshooting

15

Appendix A - Command cheat sheet

16

Appendix B - The production compose file

16

Caddyfile

18

1. How the deployment works

FleetOps is not one program but a small set of cooperating services. Each one runs in its own Docker **container**, built from a Docker **image**. In production all of them run together on a single Oracle Cloud virtual machine, managed by **Docker Compose** from one file (`deploy/docker-compose.prod.yml`). You never start containers one by one: a single command pulls the right image versions and starts, wires up and health-checks the whole stack.

The process has three stages that you repeat for every release:

Stage	Where it happens	What it does
1. Build & publish	Your PC (or GitHub Actions)	Builds the two FleetOps images (backend, frontend) for both Intel/AMD and ARM CPUs and pushes them to Docker Hub with a version tag such as <code>1.0.0</code> .
2. Pull	The OCI virtual machine	<code>docker compose pull</code> downloads the tagged FleetOps images plus the official Postgres, Redis and Caddy images.
3. Run	The OCI virtual machine	<code>docker compose up -d</code> starts or replaces containers. Data lives in named volumes, so it survives restarts and upgrades. Caddy serves the site over HTTPS on your domain.

The repository already contains everything the server needs in the `deploy/` folder:

File	Purpose
<code>deploy/docker-compose.prod.yml</code>	The production stack: 5 services, private network, volumes, health checks, log rotation.
<code>deploy/Caddyfile</code>	HTTPS reverse proxy. Gets and renews Let's Encrypt certificates automatically.
<code>deploy/.env.example</code>	Template for the secrets and settings file (<code>.env</code>) that lives only on the server.
<code>deploy/scripts/deploy.sh</code>	One-command release: backup, pull, restart, wait until healthy.
<code>deploy/scripts/backup.sh</code>	Compressed PostgreSQL backup, keeps 14 days. Run daily from cron.

Oracle VM. This guide uses an **Oracle Cloud Infrastructure (OCI) Compute** virtual machine, because that is what makes the system reachable on the internet. If you meant an Oracle **VirtualBox** VM on your own computer, sections 6 to 9 still apply (install Ubuntu, Docker, run the same compose file), but you would also need a public IP and port forwarding on your router, which is not recommended for production.

2. Images and containers

In production there are 5 containers, built from 5 images. Only 2 of those images are yours (built from this repository and published to Docker Hub). The other 3 are official images that Docker Hub already hosts.

2.1 Images you build and publish (2)

Image (Docker Hub)	Built from	Contains	Size (approx.)
<code><you>/fleetops-backend:<version></code>	<code>backend/Dockerfile</code>	Spring Boot API (Java 21 JRE on Alpine). Runs Flyway database migrations on start-up.	~200 MB
<code><you>/fleetops-frontend:<version></code>	<code>frontend/Dockerfile</code>	Compiled Angular app served by nginx. nginx also forwards <code>/api/</code> to the backend.	~25 MB

2.2 Official images pulled as-is (3)

Image	Role	Exposed to internet?
<code>postgres:16-alpine</code>	Database. Data in the <code>postgres_data</code> volume.	No, private network only
<code>redis:7-alpine</code>	Refresh-token / session store (password protected, persisted to disk).	No, private network only
<code>caddy:2-alpine</code>	HTTPS entry point and reverse proxy. Handles certificates.	Yes: ports 80 and 443

2.3 Build-only images (never run in production)

Each Dockerfile uses a **multi-stage build**: a heavy toolchain image compiles the code, then only the result is copied into a small runtime image. The toolchain images stay on the build machine.

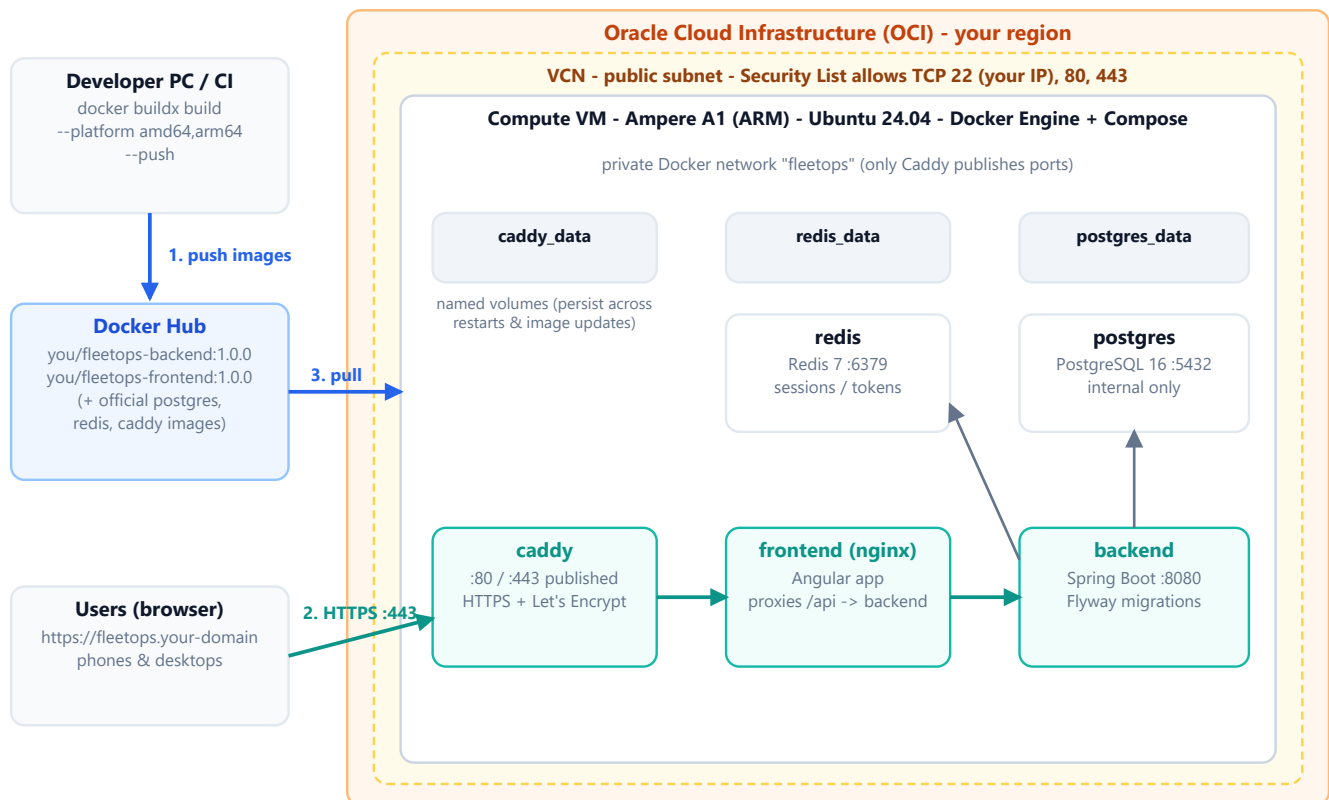
Build stage image	Used for	Final runtime image
<code>maven:3.9-eclipse-temurin-21</code>	Compiling the backend into a JAR	<code>eclipse-temurin:21-jre-alpine</code>
<code>node:20-alpine</code>	<code>npm ci</code> + <code>ng build</code> of the frontend	<code>nginx:1.25-alpine</code>

2.4 Local vs production

	Local (<code>docker-compose.yml</code>)	Production (<code>deploy/docker-compose.prod.yml</code>)
Containers	4: <code>postgres</code> , <code>redis</code> , <code>backend</code> , <code>frontend</code>	5: <code>postgres</code> , <code>redis</code> , <code>backend</code> , <code>frontend</code> , <code>caddy</code>
Images	Built on your machine (<code>build:</code>)	Pulled from Docker Hub by version tag (<code>image:</code>)
Ports open	4200, 8080, 5432, 6379 (convenient for development)	Only 80 and 443 (Caddy)
HTTPS	No	Yes, automatic Let's Encrypt certificates
Secrets	Hard-coded development values	<code>.env</code> file on the server only (chmod 600)
Swagger / API docs	On	Off

Every base image is multi-architecture. All seven base images (`eclipse-temurin`, `maven`, `node`, `nginx`, `postgres`, `redis`, `caddy`) publish both `linux/amd64` and `linux/arm64` variants (checked on 3 October 2026), so FleetOps runs on Oracle's free ARM (Ampere A1) machines.

3. Architecture



3.1 Request flow

1. The browser opens `https://your-domain`. DNS resolves the domain to the VM's reserved public IP.
2. OCI's **Security List** lets TCP 80/443 through to the VM; the VM's own firewall (iptables) allows them too.
3. **Caddy** terminates HTTPS with a Let's Encrypt certificate and forwards the request to the **frontend** container.
4. The frontend's **nginx** returns the Angular app for normal pages, and forwards anything under `/api/` to the **backend** container on port 8080.
5. The **backend** reads and writes **PostgreSQL** and stores refresh tokens in **Redis**. Neither is reachable from outside the Docker network.

3.2 Ports

Port	Where	Open to
22/tcp	VM (SSH)	Your own IP address only
80/tcp	Caddy	Everyone (HTTP, redirected to HTTPS; used for certificate issuance)
443/tcp + 443/udp	Caddy	Everyone (HTTPS, HTTP/3)
8080, 5432, 6379	backend, postgres, redis	Nobody outside the private Docker network

4. Prerequisites and costs

You need	Notes
Oracle Cloud account	Free Tier sign-up at cloud.oracle.com (credit card needed for identity checks). Pick a home region close to your users, e.g. South Africa Central (Johannesburg) . The home region cannot be changed later.
Docker Hub account	hub.docker.com . Free accounts allow unlimited public repositories but only a limited number of private ones; check current limits at docker.com/pricing .
A domain name	Any registrar (e.g. a .co.za domain). You must be able to create a DNS A record .
Build machine	Docker Desktop with buildx (already installed on your PC), or GitHub Actions.
SSH key pair	Generate with <code>ssh-keygen -t ed25519</code> if you do not have one.

4.1 Recommended VM size

Setting	Recommended	Why
Shape	VM.Standard.A1.Flex (Ampere ARM)	Part of OCI Always Free; far more memory than the free AMD micro shape.
OCPUs / memory	2 OCPU / 12 GB (minimum 1 OCPU / 6 GB)	Spring Boot + PostgreSQL + Angular build headroom.
Boot volume	100 GB	Images, database, backups and logs.
Image (OS)	Canonical Ubuntu 24.04 (aarch64)	This guide's commands assume Ubuntu; Oracle Linux notes included.

Cost: OCI Always Free currently includes Ampere A1 capacity equivalent to 4 OCPUs and 24 GB RAM, 200 GB of block storage and 10 TB/month of outbound data, so this setup can run at **R0/month** apart from the domain name. Always confirm current limits on Oracle's Free Tier page; they can change.

"Out of host capacity". Free A1 capacity is popular. If instance creation fails with this error, try another Availability Domain, try again later, or upgrade the account to Pay As You Go: you still pay nothing while you stay within the Always Free limits, and capacity is easier to get.

5. Phase 1 - Prepare a release

1. Make sure `main` builds and the app works locally (`docker compose up -d --build`, then log in and click through).
2. Choose a version number using **semantic versioning**: `MAJOR.MINOR.PATCH` (start with `1.0.0`). Bump PATCH for fixes, MINOR for new features, MAJOR for breaking changes.
3. Tag the commit so every image can be traced back to its source:

```
git checkout main && git pull
git tag -a v1.0.0 -m "FleetOps 1.0.0"
git push origin v1.0.0
```

Never deploy :latest. Production must always run an exact version tag. `latest` changes silently, makes rollbacks impossible to reason about, and different servers can end up running different code.

6. Phase 2 - Publish the images to Docker Hub

6.1 One-time Docker Hub setup

1. Sign in at hub.docker.com and create two repositories: **fleetops-backend** and **fleetops-frontend**. Public is fine: the images contain no secrets (all passwords and keys come from the server's `.env` at runtime). Use private repositories if your plan allows and you prefer the code not to be downloadable.
2. Go to **Account settings > Personal access tokens** and create two tokens: one with **Read & Write** (for your PC / CI to push) and one **Read-only** (for the server to pull). Store them in a password manager.
3. On your PC, log in with the Read & Write token (use the token as the password):

```
docker login -u <your-dockerhub-username>
```

6.2 Build for both CPU types and push

Oracle's free VMs use **ARM** processors while your PC is **x86**. An image built only for your PC fails on the server with `exec format error`. `docker buildx` builds both variants into a single multi-architecture tag; Docker Desktop emulates ARM automatically.

```
# one-time: create a builder that can produce multi-platform images
docker buildx create --name fleetops --driver docker-container --use
docker buildx inspect --bootstrap

# every release (run from the repository root)
$env:U = "<your-dockerhub-username>" # PowerShell; bash: U=<your-dockerhub-username>
$env:V = "1.0.0"

docker buildx build --platform linux/amd64,linux/arm64 `
  -t "$env:U/fleetops-backend:$env:V" --push ./backend

docker buildx build --platform linux/amd64,linux/arm64 `
  -t "$env:U/fleetops-frontend:$env:V" --push ./frontend
```

Then confirm that both architectures were published:

```
docker buildx imagetools inspect <you>/fleetops-backend:1.0.0
# expect: Platform: linux/amd64 and Platform: linux/arm64
```

Slow ARM builds? Building the ARM variant under emulation can take 10-30 minutes (Maven and the Angular compiler are CPU-heavy). Alternatives: let GitHub Actions do it (section 12), or build directly on the ARM VM with `docker build` and push from there.

Network hiccups. If a build fails with `ECONNRESET` or "Premature end of Content-Length", just run it again: both Dockerfiles cache downloaded dependencies, so a retry resumes quickly.

7. Phase 3 - Create the Oracle Cloud infrastructure

7.1 Network (VCN)

1. In the OCI Console open **Networking > Virtual cloud networks** and click **Start VCN Wizard > Create VCN with Internet Connectivity**. Name it `fleetops-vcn` and accept the defaults. This creates a public subnet, an internet gateway and route rules.
2. Open the VCN > **Security Lists > Default Security List > Add Ingress Rules** and add:

Source CIDR	Protocol	Destination port	Purpose
<your-public-ip>/32	TCP	22	SSH from your location only (replace the default 0.0.0.0/0 rule)
0.0.0.0/0	TCP	80	HTTP + Let's Encrypt validation
0.0.0.0/0	TCP	443	HTTPS
0.0.0.0/0	UDP	443	HTTP/3 (optional)

7.2 Compute instance

1. **Compute > Instances > Create instance**. Name: `fleetops-prod`.
2. **Image and shape**: Change image to **Canonical Ubuntu 24.04** (the aarch64 build is chosen automatically for ARM). Change shape to **Ampere > VM.Standard.A1.Flex** with **2 OCPUs and 12 GB** memory.
3. **Networking**: select `fleetops-vcn` and its **public subnet**; keep **Assign a public IPv4 address** on.
4. **Add SSH keys**: upload your public key (`~/ssh/id_ed25519.pub`).
5. **Boot volume**: set a custom size of **100 GB**.
6. Click **Create** and wait until the state is **Running**.

7.3 Reserve the public IP

A normal (ephemeral) public IP changes if the instance is ever recreated, which would break your DNS. Make it permanent: open the instance > **Attached VNICs > (primary VNIC) > IPv4 Addresses** > edit the address and choose **Reserved public IP > Create new**. Note this IP; DNS will point to it.

7.4 Connect

```
ssh -i ~/.ssh/id_ed25519 ubuntu@<reserved-public-ip> # Oracle Linux images use the user 'opc'
```

8. Phase 4 - Prepare the server

8.1 Updates and basics

```
sudo apt-get update && sudo apt-get -y upgrade
sudo timedatectl set-timezone Africa/Johannesburg
sudo apt-get install -y unattended-upgrades && sudo dpkg-reconfigure -plow unattended-upgrades
sudo reboot          # if the upgrade asked for it; then reconnect
```

8.2 Open the host firewall (important on OCI)

Oracle's Ubuntu images ship with **iptables** rules that reject all incoming traffic except SSH, even after the Security List allows it. Two changes are needed: allow 80/443, and remove the blanket FORWARD reject rule, which can block traffic Docker forwards to containers.

```
# 1) Allow HTTP/HTTPS. Find the line number of the REJECT rule first:
sudo iptables -L INPUT --line-numbers
# then insert the ACCEPT rules ABOVE it (replace 5 with that REJECT line number):
sudo iptables -I INPUT 5 -p tcp -m state --state NEW --dport 80 -j ACCEPT
sudo iptables -I INPUT 5 -p tcp -m state --state NEW --dport 443 -j ACCEPT
sudo iptables -I INPUT 5 -p udp --dport 443 -j ACCEPT
sudo netfilter-persistent save

# 2) Remove the FORWARD reject rule from the saved rules and reload:
sudo sed -i '/-A FORWARD -j REJECT --reject-with icmp-host-prohibited/d' /etc/iptables/rules.v4
sudo netfilter-persistent reload
```

Do not enable ufw on OCI. Oracle recommends against ufw on its images; it can lock you out of SSH. Use the iptables commands above. On **Oracle Linux** instead run: `sudo firewall-cmd --permanent --add-service=http --add-service=https && sudo firewall-cmd --reload`.

8.3 Install Docker Engine and the Compose plugin

```
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

sudo usermod -aG docker $USER      # run docker without sudo; log out and back in afterwards
sudo systemctl enable --now docker
docker version && docker compose version
```

Docker inserts its own firewall rules when it starts. If you change iptables later, run `sudo systemctl restart docker` afterwards so Docker's rules are re-applied.

9. Phase 5 - Point your domain at the server

1. At your DNS provider create an **A record**: name `fleetops` (or `@` for the bare domain), value = the **reserved public IP**, TTL 300.
2. Optionally add `www` as another A record (the Caddyfile redirects `www` to the main name).
3. Wait for propagation, then check from your PC: `nslookup fleetops.your-domain.co.za` must return the reserved IP.

Order matters. Create the DNS record **before** starting Caddy. Caddy requests the certificate on first start, and Let's Encrypt can only validate the domain once it resolves to your server and port 80 is open.

10. Phase 6 - Deploy FleetOps

10.1 Copy the deployment files to the server

```
# from your PC, in the repository root
scp -r deploy ubuntu@<reserved-public-ip>:~/fleetops

# on the server
cd ~/fleetops && chmod +x scripts/*.sh && ls
# Caddyfile docker-compose.prod.yml .env.example scripts/
```

(Alternatively `git clone` the repository on the server and work from its `deploy/` folder; then a `git pull` updates the compose file and scripts.)

10.2 Create the secrets file

```
cp .env.example .env && chmod 600 .env
openssl rand -base64 32 | tr -d '/+=' | cut -c1-32      # use for DB_PASSWORD
openssl rand -base64 32 | tr -d '/+=' | cut -c1-32      # use for REDIS_PASSWORD
openssl rand -base64 64 | tr -d '\n'; echo             # use for JWT_SECRET
nano .env      # fill in DOCKERHUB_USER, FLEETOPS_VERSION, DOMAIN, ACME_EMAIL and the three secrets
```

Variable	Example	Notes
<code>DOCKERHUB_USER</code>	<code>naventra</code>	Docker Hub account that owns the two repositories.
<code>FLEETOPS_VERSION</code>	<code>1.0.0</code>	The tag to run. Changed by <code>deploy.sh <version></code> .
<code>DOMAIN</code>	<code>fleetops.example.co.za</code>	Must match the DNS A record.
<code>ACME_EMAIL</code>	<code>it@example.co.za</code>	Let's Encrypt expiry notices.
<code>DB_PASSWORD</code> , <code>REDIS_PASSWORD</code>	<code>random</code>	Set once. Postgres only applies the password when its volume is first created.
<code>JWT_SECRET</code>	<code>random, 64+ chars</code>	Signs login tokens. Changing it logs everyone out.
<code>FLEETOPS_TIMEZONE</code>	<code>Africa/Johannesburg</code>	Business time zone for entered dates and times.
<code>MAIL_*</code>	<code>SMTP settings</code>	Optional; needed only for password-reset emails.

10.3 Log in to Docker Hub on the server

```
docker login -u <your-dockerhub-username> # paste the READ-ONLY token as the password
```

Needed for private repositories, and it also lifts Docker Hub's anonymous pull rate limits.

10.4 Start the stack

```
./scripts/deploy.sh 1.0.0
# ==> Deploying FleetOps 1.0.0
# ==> Waiting for the backend to become healthy...
# ==> Backend healthy. FleetOps 1.0.0 is live.

docker compose -f docker-compose.prod.yml --env-file .env ps
```

On the first start the backend runs the Flyway migrations, creating all tables and the initial administrator account. Caddy obtains the HTTPS certificate within a minute or so.

10.5 Verify

```
curl -I https://fleetops.example.co.za # HTTP/2 200
curl https://fleetops.example.co.za/api/actuator/health # {"status":"UP"}
docker compose -f docker-compose.prod.yml --env-file .env logs --tail=50 caddy backend
```

Change the admin password immediately. The first migration creates `admin` with the documented password `Admin@123`. Log in right away, open the user menu (top right) > **Change password**, and set a strong one. Then create real accounts under **Users** and give each person their own login. If you want zero exposure, open the site through an SSH tunnel first (`ssh -L 8443:localhost:443 ...`) or deploy before creating the DNS record.

11. Phase 7 - Operating the system

11.1 Releasing a new version

1. Tag and push the new version (section 5), build and push the images (section 6.2) with the new tag, e.g. `1.1.0`.
2. On the server: `cd ~/fleetops && ./scripts/deploy.sh 1.1.0`.

`deploy.sh` backs up the database first, updates `FLEETOPS_VERSION` in `.env`, pulls the new images, recreates only the containers whose image changed, waits for the backend health check and prints its logs if it fails. Expect about 20-40 seconds of downtime for the backend restart.

11.2 Rolling back

```
./scripts/deploy.sh 1.0.0 # previous version
```

Database migrations are forward-only. If the new version added Flyway migrations, the old version may not understand the new schema. In that case restore the backup that `deploy.sh` made just before the upgrade (11.4), then deploy the old version.

11.3 Backups

Schedule the backup script daily (it keeps 14 days of compressed dumps in `~/fleetops/backups`):

```
crontab -e
# add:
0 2 * * * /home/ubuntu/fleetops/scripts/backup.sh >> /home/ubuntu/fleetops/backups/backup.log 2>&1
```

Copies on the same disk do not protect against losing the VM. Copy them off the server, for example to an OCI **Object Storage** bucket (Always Free includes 20 GB) with the OCI CLI or rclone:

```
oci os object bulk-upload --bucket-name fleetops-backups --src-dir ~/fleetops/backups --include '*.dump'
```

11.4 Restoring a backup

```
cd ~/fleetops
C="docker compose -f docker-compose.prod.yml --env-file .env"
$C stop backend
$C exec -T postgres pg_restore -U fleetops -d fleetops --clean --if-exists < backups/fleetops-YYYYMMDD-HHMMSS.dump
$C start backend
```

11.5 Logs, monitoring and housekeeping

Task	Command / approach
Follow backend logs	<code>docker compose -f docker-compose.prod.yml --env-file .env logs -f backend</code>
Container status & health	<code>docker compose -f docker-compose.prod.yml --env-file .env ps</code>
CPU / memory per container	<code>docker stats</code>
Log size	Capped automatically: 5 x 10 MB per container (set in the compose file).
Uptime alerts	Free external monitor (e.g. UptimeRobot) on <a href="https://<domain>/api/actuator/health">https://<domain>/api/actuator/health .
VM alerts	OCI Console > Observability > Alarms on CPU, memory and boot-volume usage.
Disk clean-up	<code>docker image prune -f</code> (deploy.sh does this after each successful release).
OS patches	unattended-upgrades installs security updates; reboot monthly in a quiet window.
Audit trail	In the app: Audit Log (admin) lists every change, with user and IP address.

11.6 Growing later

- **More capacity:** edit the instance shape (A1.Flex OCPUs / memory) and reboot. No code changes.
- **Managed database:** move PostgreSQL to OCI Database with PostgreSQL; set `DB_HOST` etc. in `.env` and remove the postgres service.
- **Several app servers:** put an OCI Load Balancer in front, run backend/frontend on more VMs, keep Postgres and Redis shared. Beyond that, consider Kubernetes (OCI OKE) using the same two images.

12. Automating releases with GitHub Actions (optional)

Instead of building on your PC, let GitHub build and push the images whenever you push a version tag, then deploy over SSH. Add these **repository secrets** (Settings > Secrets and variables > Actions): `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN` (Read & Write), `OCI_HOST` (reserved IP), `OCI_SSH_KEY` (a private key whose public key is in the server's `~/.ssh/authorized_keys`).

```
# .github/workflows/release.yml
name: Release
on:
  push:
    tags: ['v*']

jobs:
  images:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        app: [backend, frontend]
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
      - uses: docker/build-push-action@v6
      with:
        username: ${ secrets.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN }
      - uses: docker/build-push-action@v6
      with:
        context: ./${ matrix.app }
        platforms: linux/amd64,linux/arm64
        push: true
        tags: ${ secrets.DOCKERHUB_USERNAME }/fleetops-${ matrix.app }:${ github.ref_name }
        cache-from: type=gha,scope=${ matrix.app }
        cache-to: type=gha,mode=max,scope=${ matrix.app }

  deploy:
    needs: images
    runs-on: ubuntu-latest
    environment: production # add a required reviewer here for a manual approval gate
    steps:
      - uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.OCI_HOST }
        username: ubuntu
        key: ${ secrets.OCI_SSH_KEY }
        script: cd ~/fleetops && ./scripts/deploy.sh ${ github.ref_name }
```

Tag format. With this workflow the Docker tag equals the Git tag (e.g. `v1.1.0`). Use the same form in `FLEETOPS_VERSION`, or strip the `v` in the workflow if you prefer plain `1.1.0`. The SSH deploy step needs port 22 open to GitHub's runners; if you restrict SSH to your own IP, keep the `images` job and run `deploy.sh` yourself.

13. Security checklist

Done	Item
☐	Admin password changed from <code>Admin@123</code> ; every person has their own account; unused accounts disabled.
☐	<code>.env</code> has strong random <code>DB_PASSWORD</code> , <code>REDIS_PASSWORD</code> , <code>JWT_SECRET</code> ; <code>chmod 600 .env</code> ; never committed.
☐	Security List: SSH (22) limited to your IP; only 80/443 open to the world.
☐	Postgres and Redis have no published ports (true in the provided compose file).
☐	HTTPS works and HTTP redirects to HTTPS (Caddy does this automatically).
☐	Swagger/API docs disabled in production (set in the compose file).
☐	Server pulls with a read-only Docker Hub token; CI uses a separate read/write token.
☐	Daily backups run and are copied off the VM; a restore has been tested at least once.
☐	Unattended security upgrades enabled; monthly reboot scheduled.
☐	Uptime monitor and OCI alarms configured.
☐	Images are pinned to a version tag, never <code>latest</code> .

14. Troubleshooting

Symptom	Likely cause	Fix
Browser times out on the domain	Security List or host iptables blocking 80/443; DNS not pointing at the VM	Check section 7.1 and 8.2; <code>nslookup <domain></code> ; from the VM <code>curl -I http://localhost</code> .
Caddy log: certificate / challenge failed	DNS not propagated yet, or port 80 blocked	Fix DNS/ports, then ... <code>restart caddy</code> . Caddy retries automatically.
<code>502 Bad Gateway</code>	frontend or backend not running/healthy	... <code>ps</code> and ... <code>logs backend frontend</code> .
<code>exec format error</code>	Image built only for amd64, server is ARM	Rebuild with <code>--platform linux/amd64,linux/arm64</code> (6.2).
Backend restarts: password authentication failed	<code>DB_PASSWORD</code> changed after the volume was created	Set it back, or <code>ALTER USER fleetops PASSWORD '...'</code> inside postgres. Postgres ignores the variable on an existing volume.
Everyone logged out after deploy	<code>JWT_SECRET</code> changed, or Redis data lost	Keep <code>JWT_SECRET</code> stable; Redis persists to the <code>redis_data</code> volume.
Times shown 2 hours off	Wrong business time zone	Set <code>FLEETOPS_TIMEZONE</code> in <code>.env</code> , redeploy.
"Out of host capacity" creating the VM	Free A1 capacity exhausted in that AD	Other AD, retry later, or upgrade to Pay As You Go (still free within limits).
<code>pull access denied</code>	Private repository and not logged in, or wrong <code>DOCKERHUB_USER</code> /tag	<code>docker login</code> on the server; check <code>.env</code> .
Disk full	Old images / logs / backups	<code>docker system df</code> , <code>docker image prune -a</code> , check backups/.

Appendix A - Command cheat sheet

```
# --- on your PC (each release) ---
git tag -a v1.1.0 -m "FleetOps 1.1.0" && git push origin v1.1.0
docker buildx build --platform linux/amd64,linux/arm64 -t <you>/fleetops-backend:1.1.0 --push ./backend
docker buildx build --platform linux/amd64,linux/arm64 -t <you>/fleetops-frontend:1.1.0 --push ./frontend

# --- on the server ---
cd ~/fleetops
./scripts/deploy.sh 1.1.0           # release (backup + pull + restart + health check)
./scripts/deploy.sh 1.0.0          # roll back
./scripts/backup.sh                # manual backup
C="docker compose -f docker-compose.prod.yml --env-file .env"
$C ps                              # status
$C logs -f backend                 # logs
$C restart caddy                   # restart one service
$C down                            # stop everything (volumes/data are kept)
```

Appendix B - The production compose file

For reference, the file shipped in `deploy/docker-compose.prod.yml` (the repository copy is authoritative):

```
# FleetOps production stack (single VM).
# Usage on the server: docker compose -f docker-compose.prod.yml --env-file .env up -d
#
# Only Caddy publishes ports (80/443). Postgres, Redis, the backend and the frontend
# talk over the private "fleetops" network and are not reachable from the internet.

name: fleetops

x-logging: &logging
driver: json-file
options:
  max-size: "10m"
  max-file: "5"

services:
  postgres:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME} -d ${DB_NAME}"]
      interval: 10s
      timeout: 5s
      retries: 10
    networks: [fleetops]
  logging: *logging
```

```
redis:
  image: redis:7-alpine
  restart: unless-stopped
  command: ["redis-server", "--requirepass", "${REDIS_PASSWORD}", "--appendonly", "yes"]
  volumes:
    - redis_data:/data
  healthcheck:
    test: ["CMD-SHELL", "redis-cli -a \"${REDIS_PASSWORD}\" --no-auth-warning ping | grep -q PONG"]
    interval: 10s
    timeout: 5s
    retries: 10
  environment:
    REDIS_PASSWORD: ${REDIS_PASSWORD}
  networks: [fleetops]
  logging: *logging
backend:
  image: ${DOCKERHUB_USER}/fleetops-backend:${FLEETOPS_VERSION}
  restart: unless-stopped
  environment:
    SPRING_PROFILES_ACTIVE: prod
    DB_HOST: postgres
    DB_PORT: 5432
    DB_NAME: ${DB_NAME}
    DB_USERNAME: ${DB_USERNAME}
    DB_PASSWORD: ${DB_PASSWORD}
    REDIS_HOST: redis
    REDIS_PORT: 6379
    REDIS_PASSWORD: ${REDIS_PASSWORD}
    JWT_SECRET: ${JWT_SECRET}
    CORS_ORIGINS: https://${DOMAIN}
    FLEETOPS_TIMEZONE: ${FLEETOPS_TIMEZONE:-Africa/Johannesburg}
    MAIL_HOST: ${MAIL_HOST:-}
    MAIL_PORT: ${MAIL_PORT:-587}
    MAIL_USERNAME: ${MAIL_USERNAME:-}
    MAIL_PASSWORD: ${MAIL_PASSWORD:-}
    UPLOAD_PATH: /app/uploads
    # API docs are for development; keep them off on the public internet
    SPRINGDOC_API_DOCS_ENABLED: "false"
    SPRINGDOC_SWAGGER_UI_ENABLED: "false"
    # Let the JVM size its heap from the container's memory
    JAVA_TOOL_OPTIONS: "-XX:MaxRAMPercentage=60"
  volumes:
    - uploads:/app/uploads
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:8080/api/actuator/health | grep -q UP"]
    interval: 15s
    timeout: 5s
    retries: 10
    start_period: 60s
  networks: [fleetops]
  logging: *logging

frontend:
  image: ${DOCKERHUB_USER}/fleetops-frontend:${FLEETOPS_VERSION}
  restart: unless-stopped
  depends_on:
    backend:
      condition: service_healthy
```

```

    networks: [fleetops]
    logging: *logging

caddy:
  image: caddy:2-alpine
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
    - "443:443/udp"
  environment:
    DOMAIN: ${DOMAIN}
    ACME_EMAIL: ${ACME_EMAIL}
  volumes:
    - ./Caddyfile:/etc/caddy/Caddyfile:ro
    - caddy_data:/data
    - caddy_config:/config
  depends_on:
    - frontend
  networks: [fleetops]
  logging: *logging

networks:
  fleetops:

volumes:
  postgres_data:
  redis_data:
  uploads:
  caddy_data:
  caddy_config:

```

Caddyfile

```

# Caddy terminates HTTPS (free Let's Encrypt certificates, renewed automatically)
# and forwards everything to the frontend container. The frontend's nginx serves the
# Angular app and proxies /api/ to the backend on the private Docker network.
{
  email ${ACME_EMAIL}
}

${DOMAIN} {
  encode zstd gzip

  header {
    Strict-Transport-Security "max-age=31536000; includeSubDomains"
    X-Content-Type-Options "nosniff"
    Referrer-Policy "strict-origin-when-cross-origin"
    -Server
  }
  reverse_proxy frontend:80
}

# Redirect www to the bare domain (remove if you don't use www)
www.${DOMAIN} {
  redir https://{${DOMAIN}}{uri} permanent
}

```